3.modelling (exercises)

Joao Lopes

2026-03-25

Contents

1. BUILD A MODEL **3**
1.1. LOOK AT DATA . . . 3
1.2. FIT A MODEL . . . 3
1.3. USE MODEL TO PREDICT . . . 3

2. BUILD A MODEL USING WORKFLOW **4**
2.1. LOOK AT DATA . . . 4
2.2. SPLIT DATA . . . 4
2.3. CREATE MODEL . . . 4
2.4. CREATE RECIPE . . . 4
2.5. CREATE WORKFLOW . . . 4
2.6. FIT A MODEL . . . 4
2.7. EVALUATE MODEL . . . 5

3. BUILD VARIOUS MODELS USING WORKFLOW **6**
3.1. LOOK AT DATA . . . 6
3.2. SPLIT DATA . . . 6
3.3. CREATE WORKFLOW AND FIT MODEL 1 . . . 6
3.4. CREATE WORKFLOW AND FIT MODEL 2 . . . 6
3.5. EVALUATE MODEL . . . 6

1. BUILD A MODEL

1.1. LOOK AT DATA

[no exercises]

1.2. FIT A MODEL

a) Evaluate the following model using function glance(). Evaluate it also using function augment() and rsq() from tidymodels. What are the possible problems of using the same data to fit and to evaluate the model?

```
set.seed(123)
lev_cut = c("Fair", "Good", "Very Good", "Premium", "Ideal")
diamonds_data <- diamonds |>
  mutate(
    log_price = log(price),
    log_carat = log(carat),
    cut = fct(as.character(cut), levels = lev_cut),
    .keep = "used"
  ) |>
  slice_sample(n = 1000)
lm_fit <- linear_reg() |>
  fit(log_price ~ log_carat + cut, data = diamonds_data)
```

b) How can you avoid overestimating the evaluation of the model? Consider taking a new sample of the data.

c) Using the same fitted model, build by hand a plot showing how the residuals vary a long the values of the predictors.

1.3. USE MODEL TO PREDICT

a) Using the model from the previous section, predict the price of diamonds with 1, 2 and 3 carat for cuts "Fair", "Very Good" and "Ideal". Build a plot of the predictions using a confidence interval of 90%.

---

2. BUILD A MODEL USING WORKFLOW

2.1. LOOK AT DATA

[no exercises]

2.2. SPLIT DATA

[no exercises]

2.3. CREATE MODEL

[no exercises]

2.4. CREATE RECIPE

[no exercises]

2.5. CREATE WORKFLOW

a) Considering the following workflow, build a new workflow without the steps of the recipe. Consider replacing the function add_recipe() with add_formula().

```
glm_mod <- logistic_reg() |>
  set_engine("glm")
glm_rec <-
  recipe(arr_delay ~ ., data = train_data) |>
  step_date(date, features = c("dow", "month")) |>
  step_holiday(date, holidays = timeDate::listHolidays("US")) |>
  step_rm(date) |>
  step_dummy(all_nominal_predictors()) |>
  step_zv(all_predictors())
glm_wflow <-
  workflow() |>
  add_model(glm_mod) |>
  add_recipe(glm_rec)
```

2.6. FIT A MODEL

a) Using the given workflow and the newly created one, fit the two models to the following data set. Consider the data split in train and test sets.

```
set.seed(123)
flight_data <-
  flights |>
  mutate(
    arr_delay = if_else(arr_delay >= 30, "late", "on_time"),
    arr_delay = fct(arr_delay, levels = c("late", "on_time")),
    date = lubridate::as_date(time_hour)
  ) |>
  select(dep_time, origin, dest, distance, carrier, date, arr_delay) |>
  drop_na() |>
  mutate(across(where(is.character), as.factor)) |>
  slice_sample(n = 10000)

set.seed(456)
```

```r
data_split <- initial_split(flight_data, prop = 3/4)
train_data <- training(data_split)
test_data  <- testing(data_split)
```

2.7. EVALUATE MODEL

a) How did the previously fitted models perform? Use the appropriate statistic.

b) Looking at the augmented data set, try to guess which probability threshold was used for predicting the classes in .pred_class?

c) Using function conf_mat(), calculate a confusion matrix for the best model. Using function tabyl(), calculate by hand a confusion matrix to the best model.

d) Use probability thresholds of 0.40 and 0.60 to predict the classes of the best model. How does the threshold value impact on the confusion matrix?

e) Using the original confusion matrix calculate the sensitivity (= TP/P) and the specificity (= TN/N).

f) Create the ROC curve and place the calculated values of sensitivity and specificity on it. Use geom_hline(), geom_vline() and geom_point().

g) Find which terms are more important to the best fitted model and plot them. Use the statistics of each term as a proxy for their importance. Try also using the estimates of the coefficients. Are the results similar? Note that term "(Intercept)" is meaningless and should be discarded.

---

3. BUILD VARIOUS MODELS USING WORKFLOW

3.1. LOOK AT DATA

[no exercises]

3.2. SPLIT DATA

[no exercises]

3.3. CREATE WORKFLOW AND FIT MODEL 1

a) Considering the following split data set and tidymodel recipe, create the respective workflow and fit a simple logistic model. Use the ROC curve and ROC area to evaluate your model.

```
hotels <-
  read_csv("https://tidymodels.org/start/case-study/hotels.csv") |>
  mutate(across(where(is.character), as.factor))

set.seed(123)
splits      <- initial_split(hotels, strata = children, prop = 0.75)
hotel_other <- training(splits)
hotel_test  <- testing(splits)

holidays <- c("AllSouls", "AshWednesday", "ChristmasEve", "Easter",
              "ChristmasDay", "GoodFriday", "NewYearsDay", "PalmSunday")
lr_recipe <-
  recipe(children ~ ., data = hotel_other) |>
  step_date(arrival_date) |>
  step_holiday(arrival_date, holidays = holidays) |>
  step_rm(arrival_date) |>
  step_dummy(all_nominal_predictors()) |>
  step_zv(all_predictors()) |>
  step_normalize(all_predictors())
```

3.4. CREATE WORKFLOW AND FIT MODEL 2

a) Replace the glm model used above with a neural network model. Consider function mlp() with engine "nnet" and mode "classification". For the hyper parameters, set hidden_units = 1 and penalty = tune(). Use the same recipe as before. To tune the model set a grid of 10 points and use the given data split. Choose the best model and evaluate it with a ROC curve and the ROC area. How confident are you about the tuning?

```
set.seed(234)
val_set <- validation_split(hotel_other, strata = children, prop = 0.80)
```

3.5. EVALUATE MODEL

a) Choose the best tuned neural network model and evaluated it using the test data created before. For the evaluation, consider the ROC curve and ROC area. Finish the analysis by plotting the most important features (i.e. predictors) of the final model. Set the importance parameter of the engine to "impurity".